

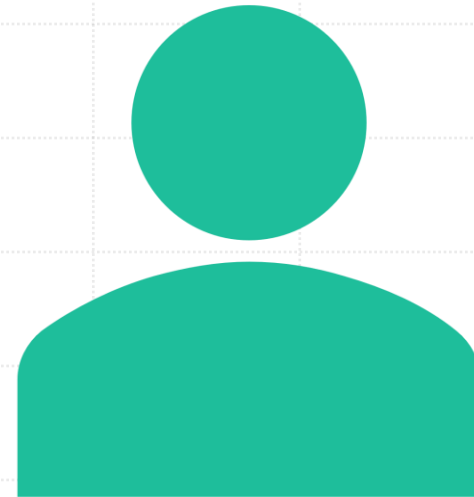
¿Por qué hablamos de
documentación y diseño?



¿Por qué hablamos de documentación y diseño?

- Un **arquitecto de software** no solo diseña sistemas, también debe **comunicar** su visión al equipo (desarrolladores, testers, líderes, stakeholders).

Si la arquitectura no está documentada, es como un edificio sin planos: nadie sabe cómo construirlo, mantenerlo o escalarlo.





Objetivo del diseño y documentación

Comunicación

- Es el idioma común entre arquitectos, desarrolladores, testers y stakeholders (usuarios, jefes, clientes).
- Un diagrama explica en 5 minutos lo que tomaría 5 páginas de texto.

Toma de decisiones informada

- La arquitectura es el resultado de **decisiones técnicas críticas**:
 - ¿Monolito o microservicios?
 - ¿Base de datos SQL o NoSQL?
 - ¿API REST o eventos con Kafka?
- Documentar estas decisiones (ej. ADR) permite justificar el “por qué”.

Mantenibilidad y evolución



Objetivo
del diseño y
documentación

- El software **no es estático**: evoluciona.
- Una buena documentación permite que en 6 meses otro equipo pueda agregar funcionalidades sin romper todo.

Niveles de documentación en arquitectura

Visión general (para negocio y stakeholders no técnicos):

Explica el “qué” y el “para qué”.

Ejemplo: un **diagrama de contexto C4** donde se ve el sistema y actores externos.

Diseño lógico (para arquitectos y desarrolladores):

Explica el “cómo funciona internamente”.

Ejemplo: **diagramas UML de componentes, secuencia y clases**.

Diseño físico (para DevOps e infraestructura):

Explica “dónde y cómo se despliega”.

Ejemplo: un **diagrama de despliegue** que muestre servidores, Kubernetes, nube, etc.

La regla de oro: “La documentación debe ser útil, no decorativa”.





Ejemplo: Sistema de e-commerce

Sin documentación:

- Un dev implementa el carrito en memoria, otro en base de datos.
- El equipo de pagos usa SOAP, el equipo de inventario usa REST.
- Resultado: integración caótica.

Con documentación:

- Se definen **componentes principales**: catálogo, carrito, pagos, usuarios.
- Se acuerda que todos los servicios expongan **REST APIs JSON**.
- Se documenta en un **ADR** que la base de datos será PostgreSQL por soporte de transacciones.
- Se crean **diagramas UML y C4** que muestran las interacciones.



Tipos de Documentos Arquitectónicos

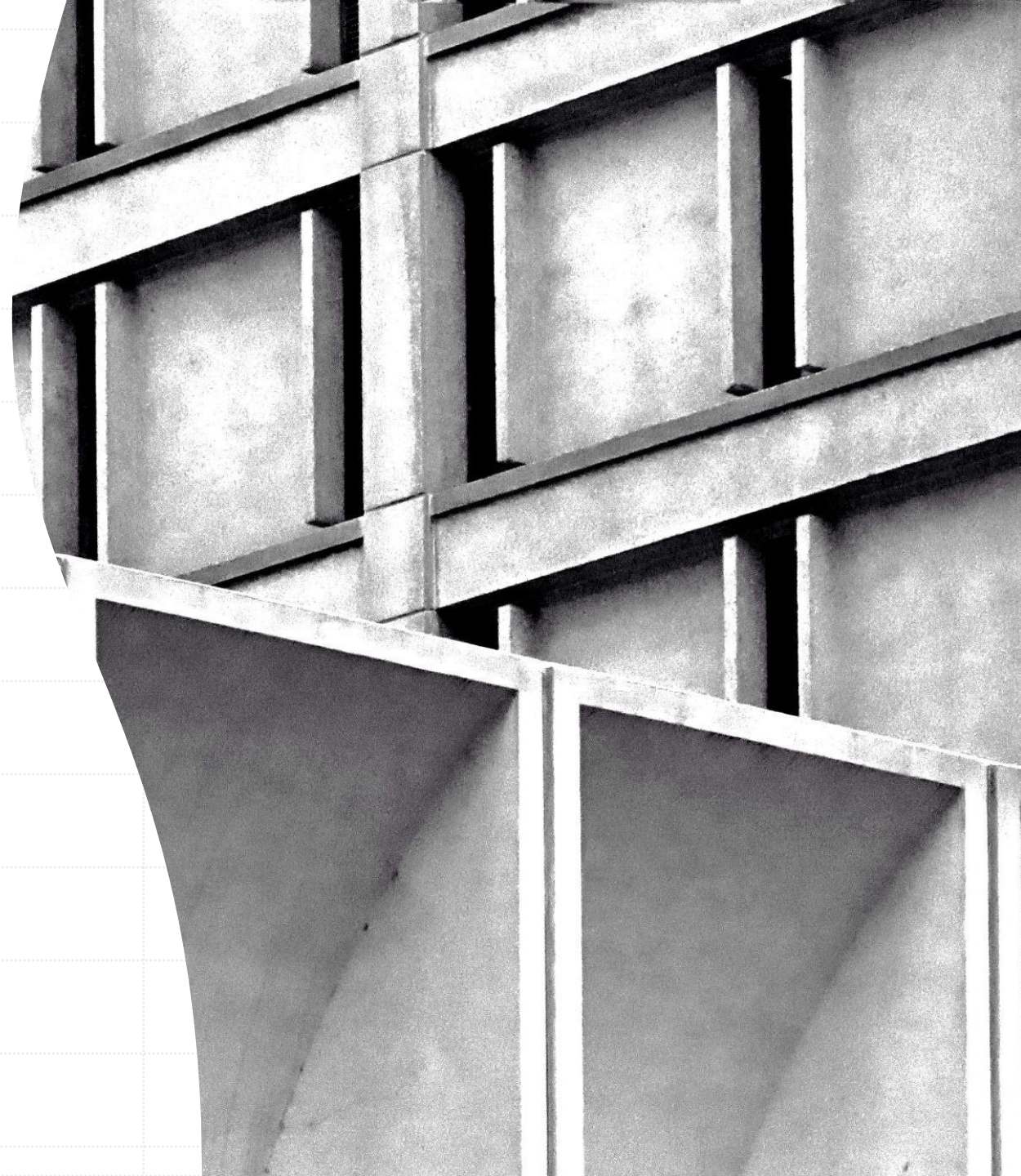
- Un arquitecto de software no escribe un solo tipo de documento, sino que adapta la documentación según el **momento del proyecto** y el **nivel de detalle requerido**.



ADR (Architecture Decision Records)

¿Qué es?

- ✓ Un **registro breve** donde se documenta una decisión arquitectónica importante. Se enfoca en el “**qué decidimos y por qué**”.



Estructura típica:



Título → “Usar Kafka como sistema de mensajería”.



Estado → Aceptado, Propuesto, Rechazado, Reemplazado.



Contexto → Descripción del problema.



Decisión → La opción elegida.



Consecuencias → Pros y contras de esa decisión.

Ejemplo

- **ADR 001 - Uso de PostgreSQL como base de datos principal**
 - ✓ **Estado:** Aceptado
 - ✓ **Contexto:** Se requiere una base de datos relacional para manejar transacciones ACID en el sistema de pagos.
 - ✓ **Decisión:** Se elige PostgreSQL por su soporte ACID, escalabilidad y costo (open source).
 - ✓ **Consecuencias:**
 - Ventaja: robustez, transacciones seguras.
 - Desventaja: curva de aprendizaje para algunos devs.





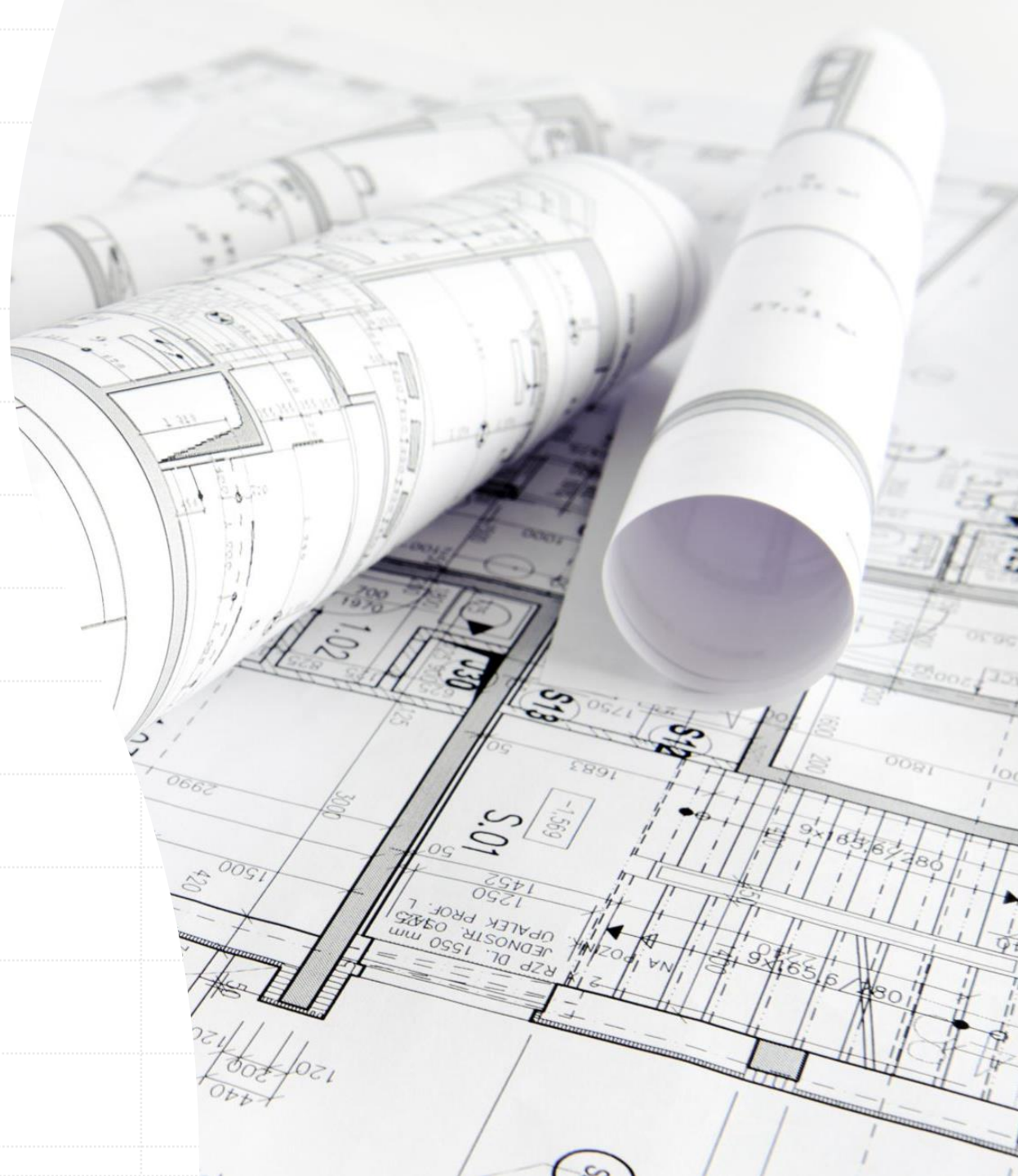
Importancia:

Permite que dentro de 6 meses alguien entienda **por qué se eligió PostgreSQL y no MySQL.**

SRS (Software Requirements Specification)

¿Qué es?

- Es el **documento formal de requisitos del sistema**.
Aunque no es estrictamente de arquitectura, es **la base para diseñar la arquitectura**.





Estructura (IEEE 830 estándar):

Introducción → propósito, alcance, definiciones.

Descripción general del sistema → contexto, actores, restricciones.

Requisitos funcionales → lo que el sistema debe hacer.

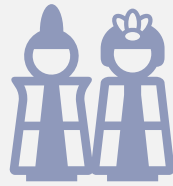
- Ejemplo: “El sistema debe permitir a un usuario autenticarse con correo y contraseña”.

Requisitos no funcionales → calidad del sistema.

- Ejemplo: “El sistema debe procesar 1000 transacciones por segundo”.

Restricciones → hardware, regulaciones, estándares.

Ejemplo en un sistema de banca digital:



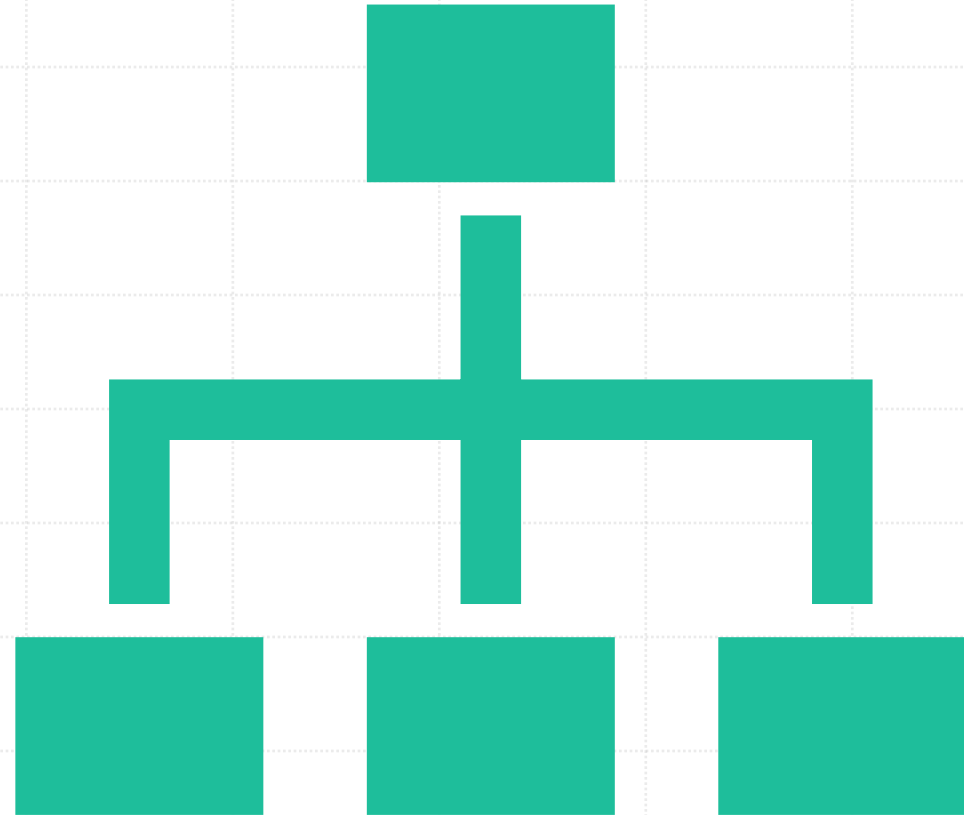
Requisito funcional: “El cliente podrá transferir dinero entre cuentas propias y de terceros”.



Requisito no funcional: “El sistema debe estar disponible 99.9% del tiempo”.

Importancia:

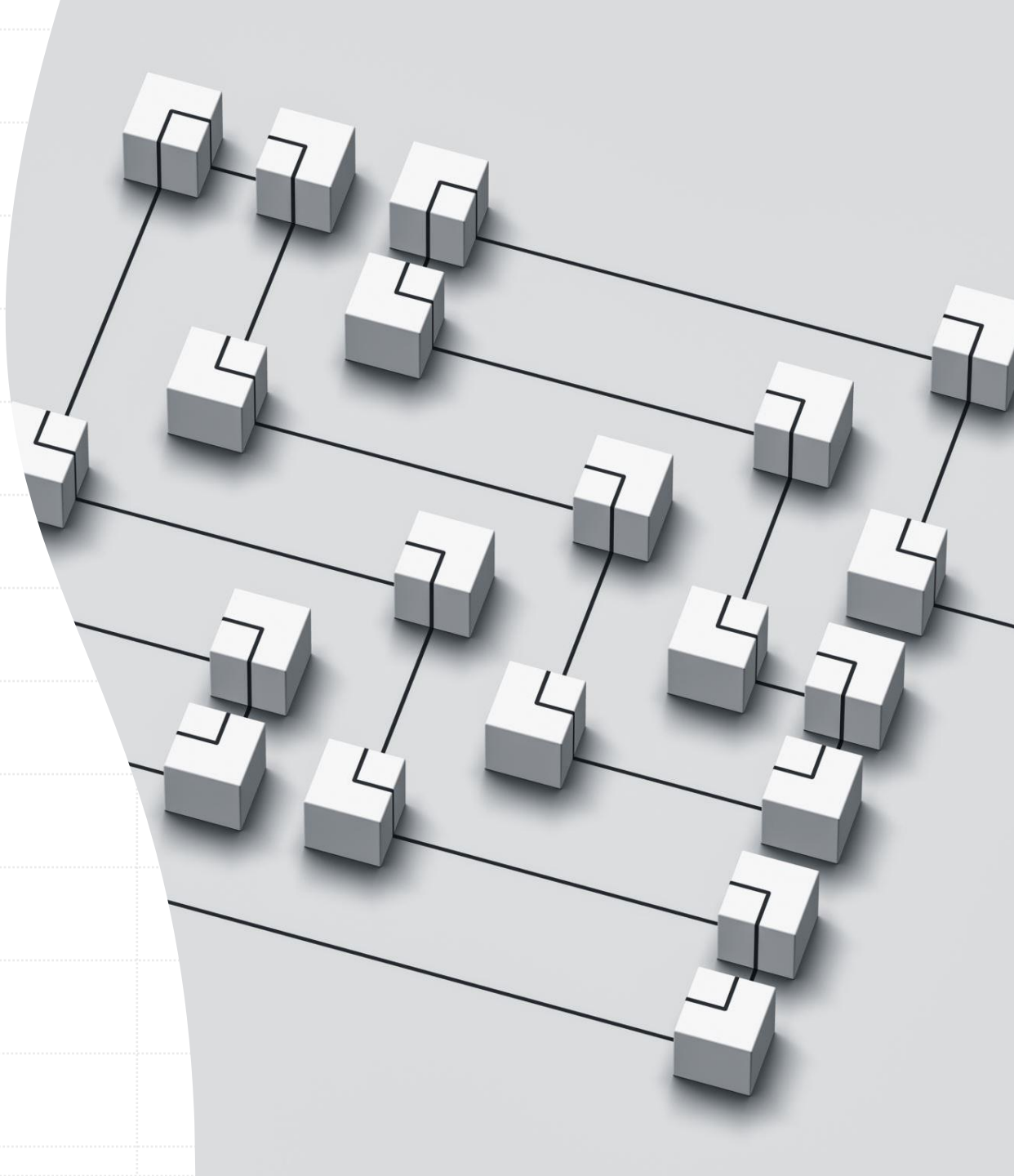
Guía al arquitecto a definir
qué **componentes y
tecnologías** necesita.



Diagramas UML (Unified Modeling Language)

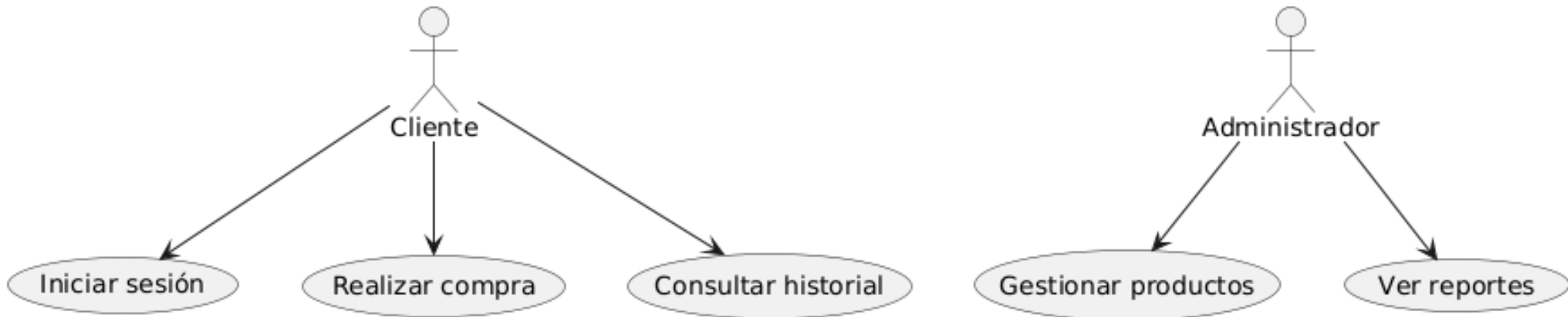
¿Qué es?

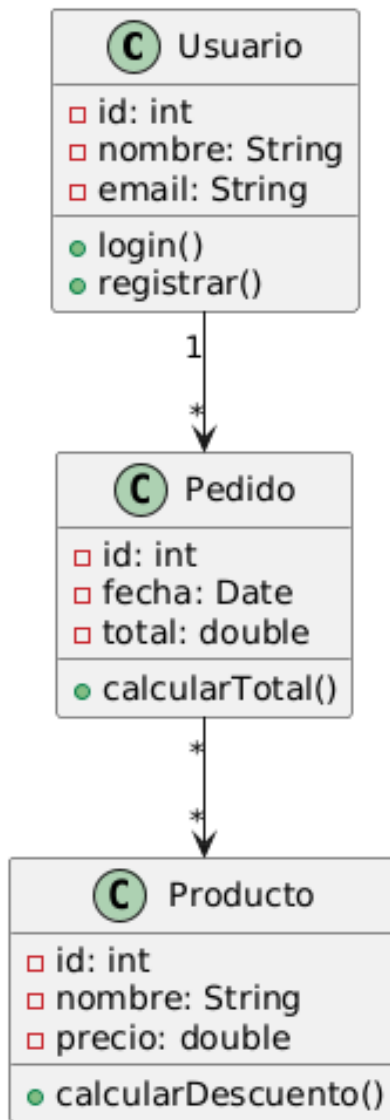
- Un **lenguaje estándar de modelado** que permite representar sistemas de software gráficamente.



Diagramas más usados en arquitectura:

- **Casos de uso** → vista funcional del sistema.



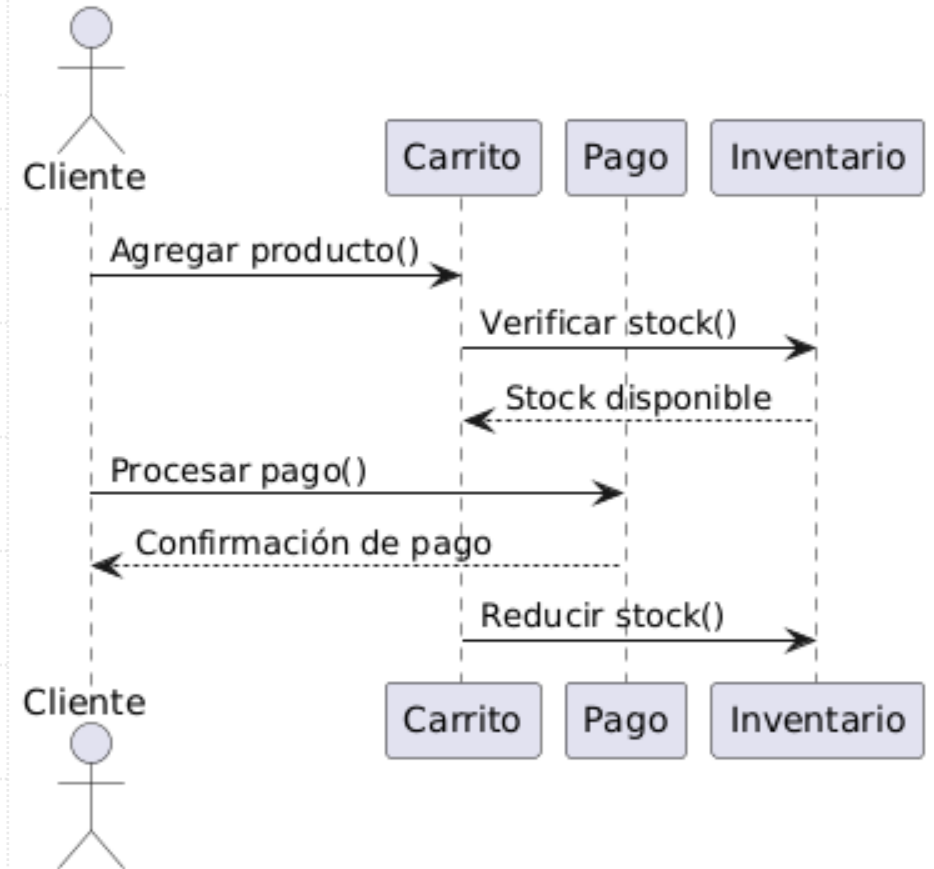


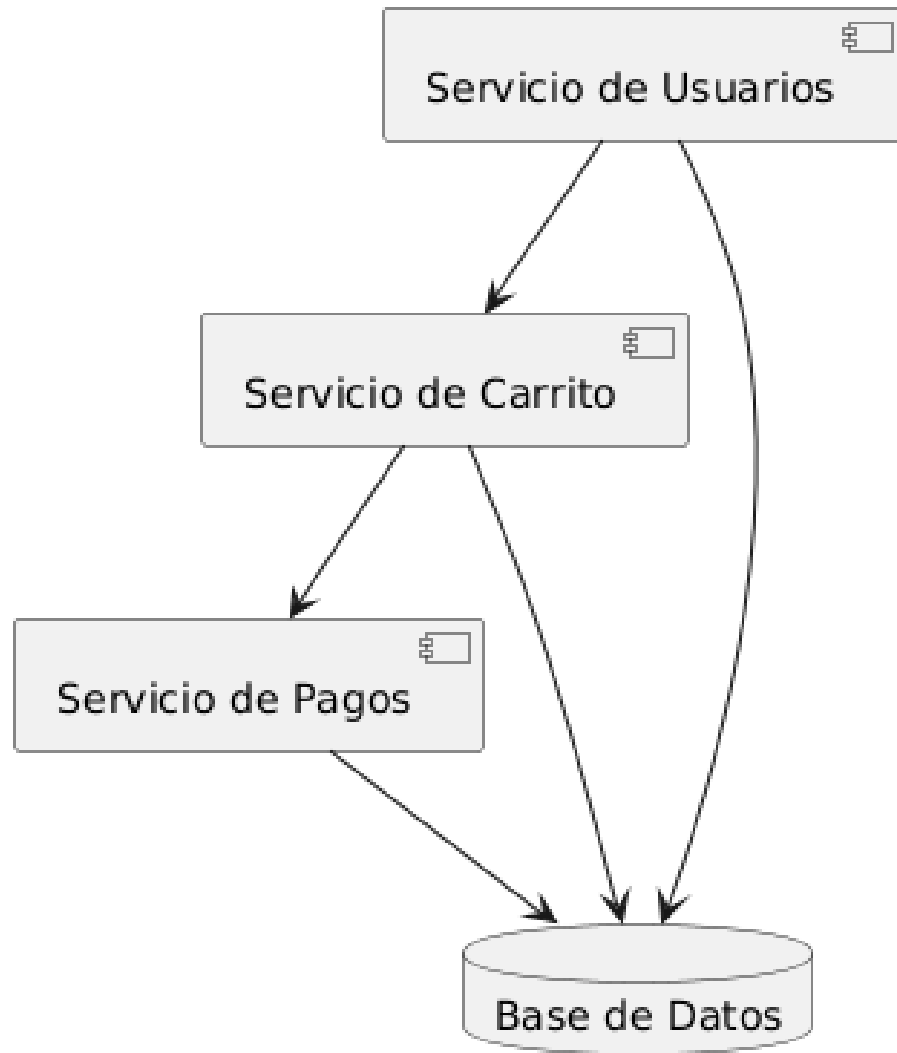
Diagramas más usados en arquitectura:

- **Clases** → vista estática de objetos y relaciones.
 - ✓ Ejemplo: Clase Usuario con atributos y métodos.

Diagramas más usados en arquitectura:

- **Secuencia** → vista dinámica de interacciones en el tiempo.
 - ✓ Ejemplo: cómo un cliente hace un pedido → Cliente → Carrito → Pago → Inventario.

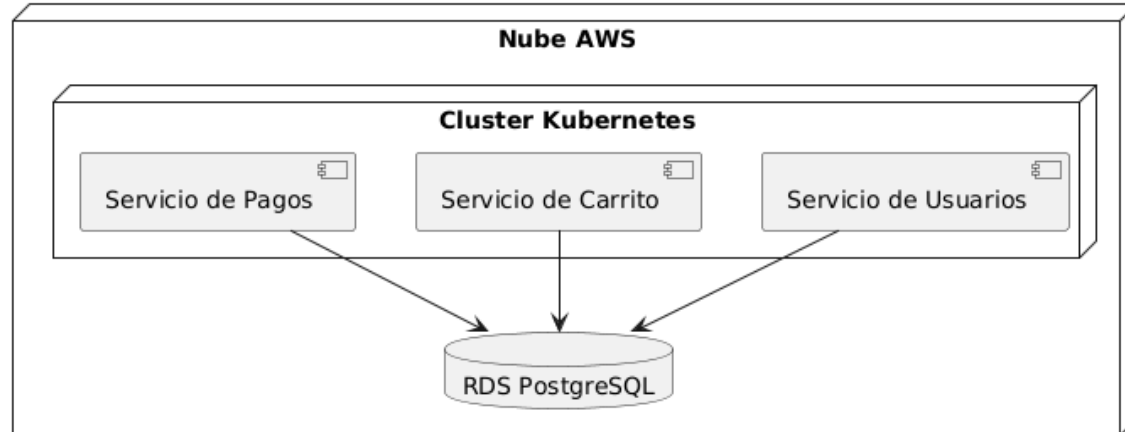




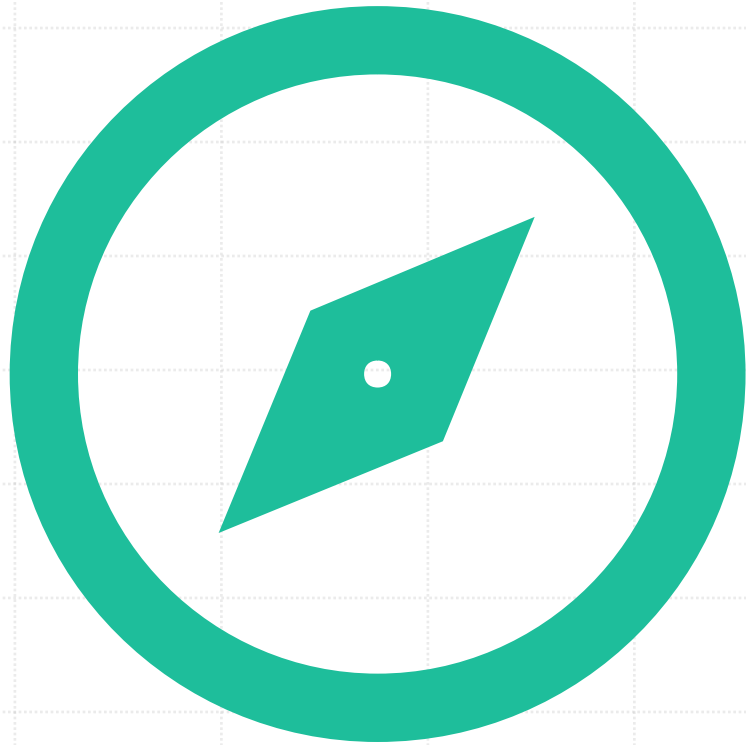
Diagramas más usados en arquitectura:

- **Componentes** → cómo se organizan los módulos del sistema.
 - ✓ Ejemplo: Servicio de Pagos, Servicio de Usuarios, Base de Datos.

Diagramas más usados en arquitectura:



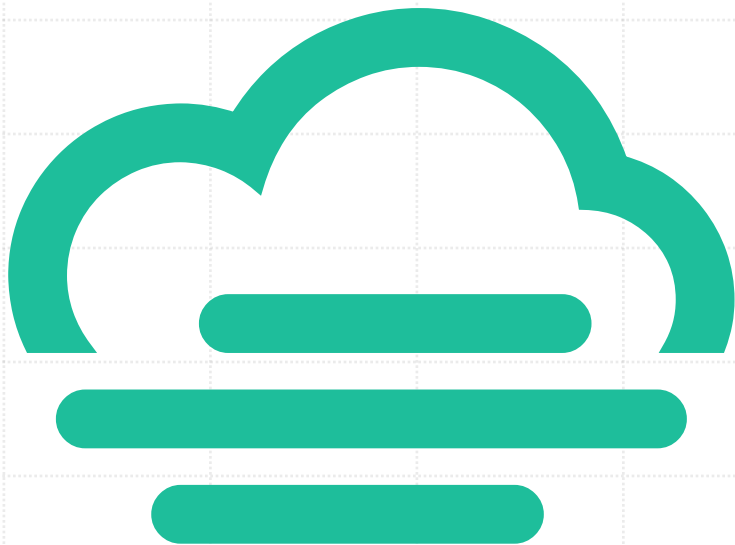
- **Despliegue** → cómo se distribuyen los componentes en servidores/nube.
 - ✓ Ejemplo: App en Kubernetes + DB en AWS RDS.
 - ✓ Estereotipos comunes:
 - ✓ <<device>> -> Hardware Fisico
 - ✓ <<executionEnvironment>> -> Maquina virtual, contenedor
 - ✓ <<artifact>> -> componente a desplegar



Otros documentos arquitectónicos complementarios

Documentos de visión / roadmaps

- ✓ Explican la **visión global** del sistema y cómo evolucionará.
- ✓ Ejemplo: “Migrar de monolito a microservicios en 3 fases durante 12 meses”.



Documento de Visión / Roadmap

- **Título:** Roadmap de Migración del Sistema E-Commerce Monolítico a Microservicios
- **Objetivo:** Evolucionar el sistema de e-commerce actual (monolito en Java EE) hacia una arquitectura de microservicios en la nube para mejorar escalabilidad y mantenibilidad.
- **Alcance:** 12 meses, dividido en 3 fases.



Documento de Visión / Roadmap

Fases:

- **Fase 1 (Meses 1-4):**
 - ✓ Identificación de dominios principales (Usuarios, Productos, Carrito, Pagos).
 - ✓ Creación de APIs REST independientes para cada dominio.
 - ✓ Migración del módulo de Usuarios como microservicio en Spring Boot.
- **Fase 2 (Meses 5-8):**
 - ✓ Migración de Productos y Carrito a microservicios.
 - ✓ Implementación de mensajería con Kafka para sincronización de inventario.
 - ✓ Pruebas de carga iniciales en Kubernetes (AWS EKS).
- **Fase 3 (Meses 9-12):**
 - ✓ Migración del módulo de Pagos con integración a pasarela externa.
 - ✓ Implementación de observabilidad (Prometheus + Grafana).
 - ✓ Configuración de CI/CD con Jenkins y SonarCloud.
- **Resultado esperado:**
Un sistema distribuido, escalable y desplegado en Kubernetes con microservicios independientes.

Especificación Técnica de Interfaces (API Contract)

The image shows the Swagger Editor interface with an OpenAPI specification for 'API de Usuarios - E-commerce'. The left pane displays the raw OpenAPI JSON, and the right pane shows the rendered API documentation.

OpenAPI Specification (Left Pane):

```
1 openapi: 3.0.1
2 info:
3   title: API de Usuarios - E-commerce
4   version: 1.0.0
5   description: API para gestión de usuarios del sistema de e-commerce
6 servers:
7   - url: https://api.ecommerce.com/v1
8 paths:
9   /usuarios:
10    post:
11      summary: Registrar un nuevo usuario
12      requestBody:
13        required: true
14        content:
15          application/json:
16            schema:
17              $ref: '#/components/schemas/Usuario'
18      responses:
19        '201':
20          description: Usuario creado
21          content:
22            application/json:
23              schema:
24                $ref: '#/components/schemas/Usuario'
25    get:
26      summary: Listar usuarios
27      responses:
28        '200':
29          description: Lista de usuarios
30          content:
31            application/json:
32              schema:
33                type: array
34                items:
35                  $ref: '#/components/schemas/Usuario'
36  /usuarios/{id}:
37    get:
38      summary: Obtener usuario por ID
39      parameters:
40        - name: id
41          in: path
42          required: true
43          schema:
44            type: string
45      responses:
46        '200':
```

Rendered API Documentation (Right Pane):

API de Usuarios - E-commerce 1.0.0 OAS 3.0

API para gestión de usuarios del sistema de e-commerce

Servers

https://api.ecommerce.com/v1

default

- POST** /usuarios Registrar un nuevo usuario
- GET** /usuarios Listar usuarios
- GET** /usuarios/{id} Obtener usuario por ID

Schemas

Usuario

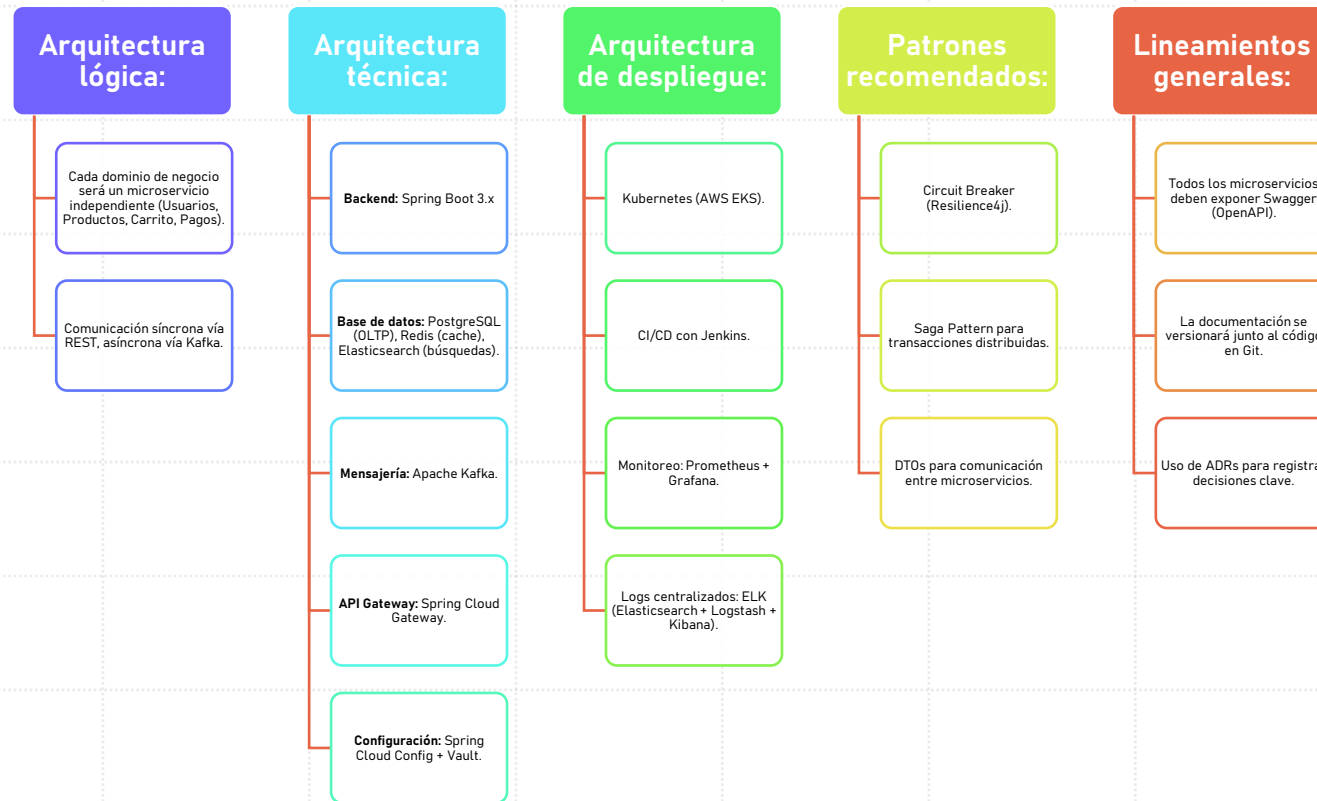
- id > [...]
- nombre > [...]
- email > [...]

Documento de Arquitectura de Referencia

Título: Arquitectura de Referencia para Microservicios en E-Commerce

Propósito: Definir una plantilla estándar que guíe el desarrollo de aplicaciones basadas en microservicios dentro de la organización.

Documento de Arquitectura de Referencia

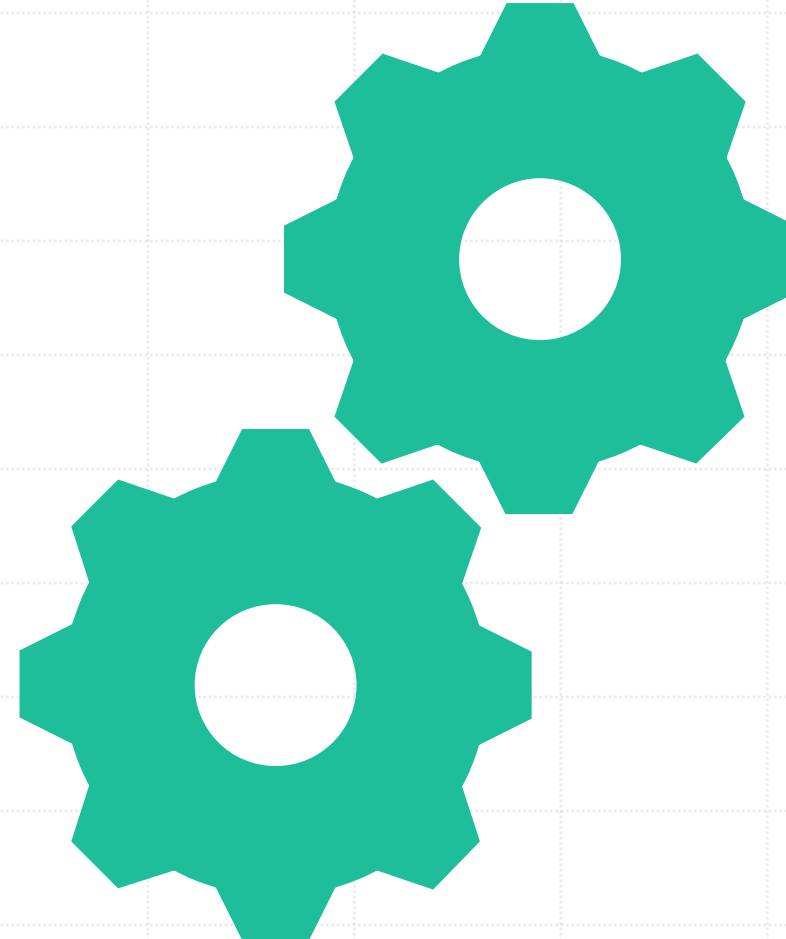


Herramientas para Documentación Arquitectónica



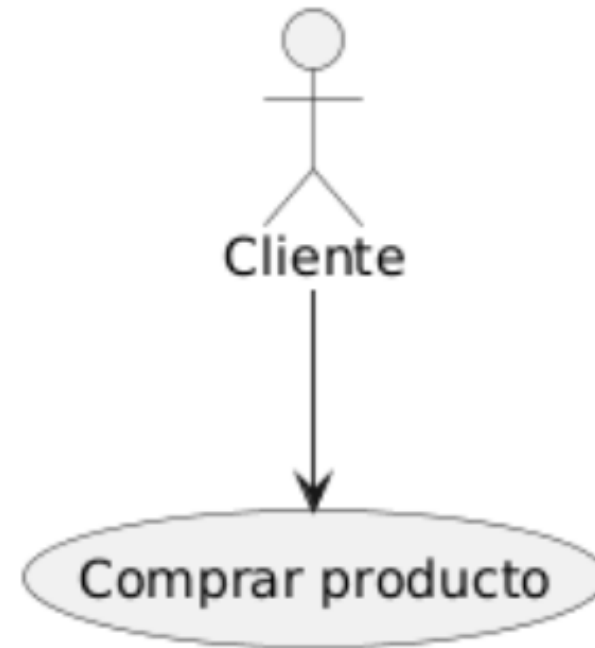
PlantUML

- **Qué es:**
Un motor que genera **diagramas UML y C4 a partir de texto**.
- **Ventajas:**
 - ✓ Ideal para proyectos versionados en Git (los diagramas son texto plano .puml).
 - ✓ Permite automatizar la generación de diagramas en CI/CD.
 - ✓ Compatible con UML y con el modelo C4 (muy usado en microservicios).



Ejemplo

```
1 @startuml
2 actor Cliente
3 Cliente --> (Comprar producto)
4 @enduml
```



Draw.io (hoy Diagrams.net)



Ventajas:

- Muy intuitiva (drag & drop).
- Se integra con Google Drive, OneDrive y GitHub.
- Exporta a PNG, SVG, PDF y también a .drawio (editable).

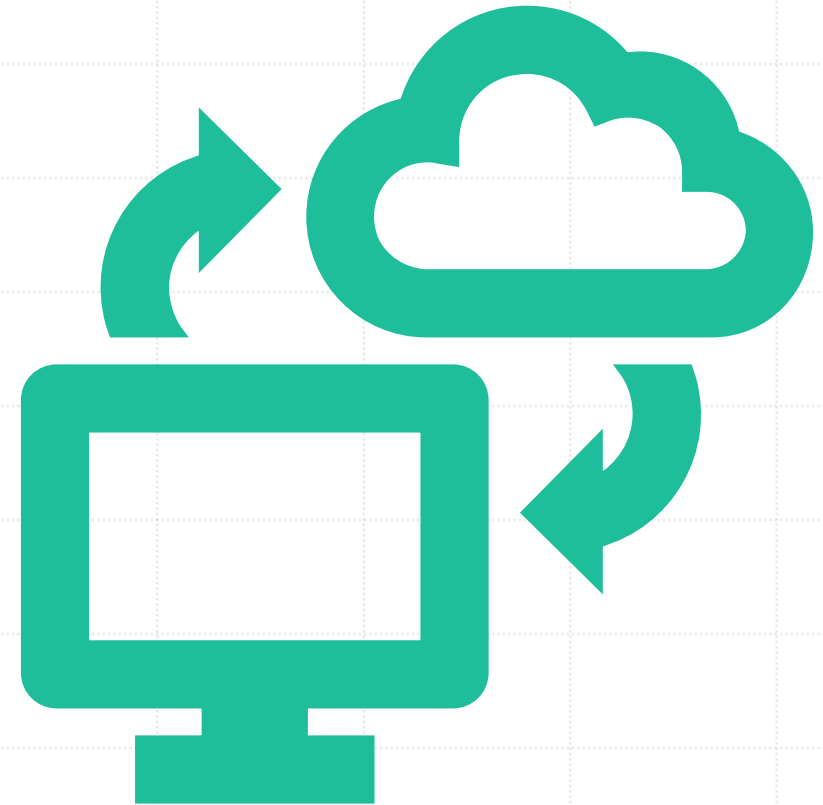
Ejemplo de uso:

- Perfecto para **explicar a stakeholders no técnicos**.
- Puedes arrastrar iconos de AWS, Azure o Kubernetes para diagramas de infraestructura.

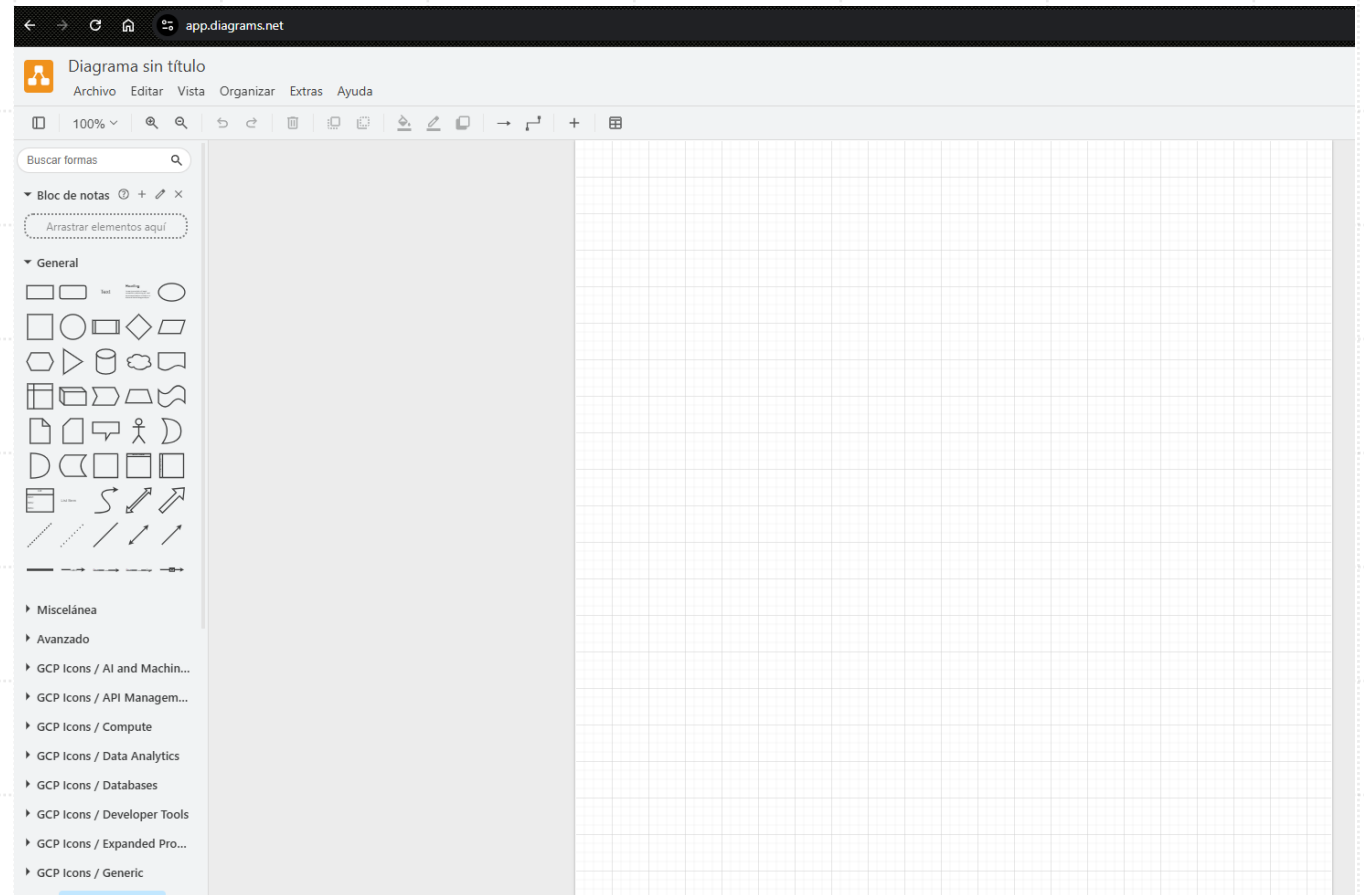
Draw.io (hoy Diagrams.net)

- **Caso práctico:**

- ✓ Arquitecto diseña el diagrama de despliegue con íconos oficiales de AWS.
- ✓ Lo guarda en `/docs/architecture/diagrams/despliegue.drawio`.



Draw.io (hoy Diagrams.net)



Lucidchart

Qué es:

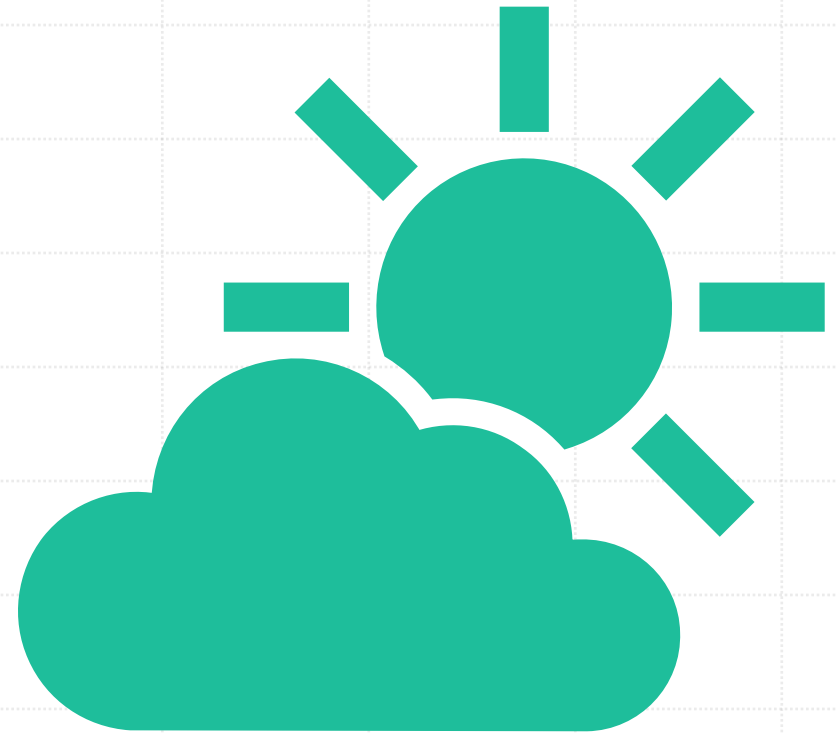
Herramienta en la nube para **diagramación colaborativa en tiempo real**.

▪ Ventajas:

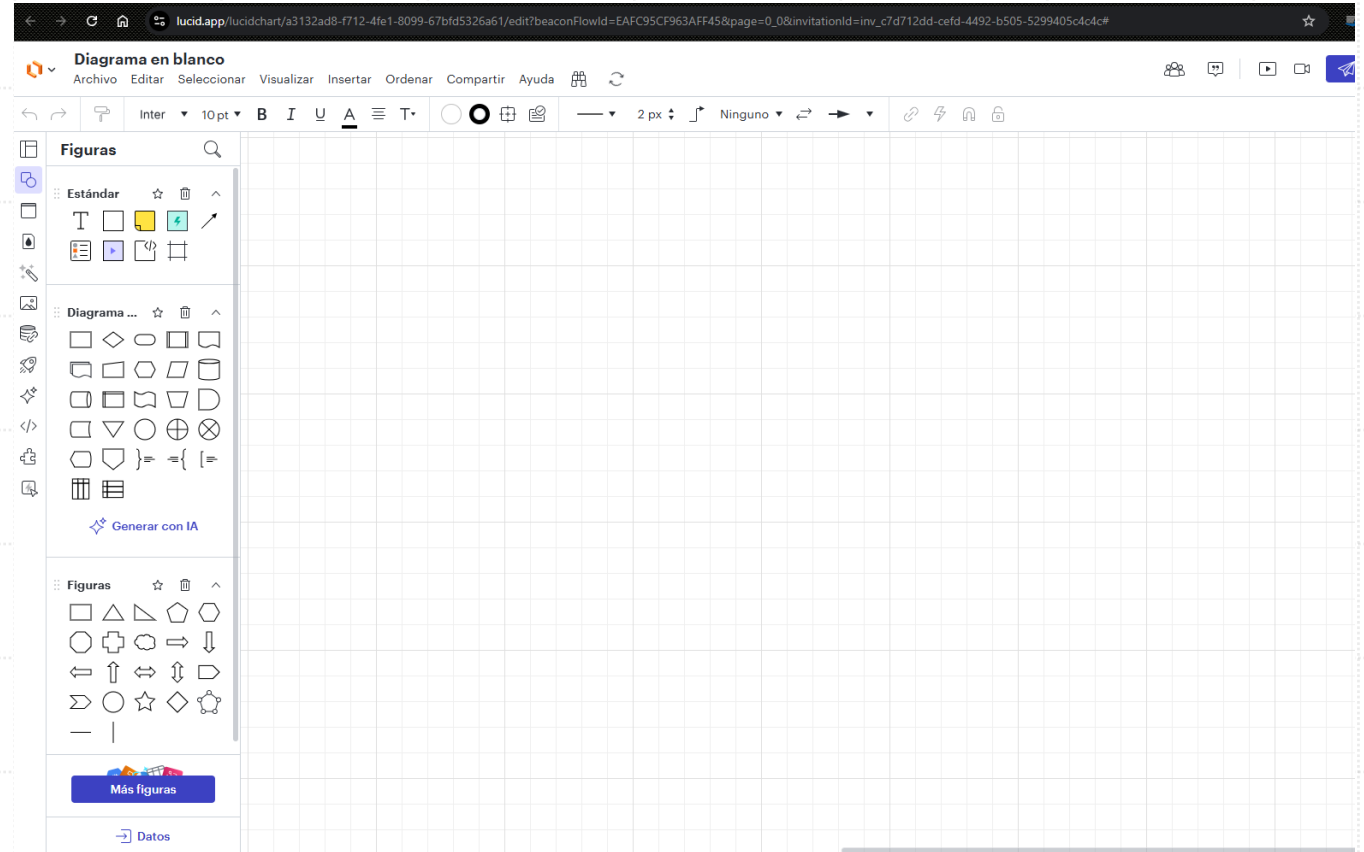
- ✓ Colaboración en equipo tipo Google Docs (varias personas editando a la vez).
- ✓ Amplia librería de plantillas (UML, C4, BPMN, infraestructura).
- ✓ Exporta a muchos formatos (Visio, PNG, PDF, SVG).

▪ Caso práctico:

- ✓ El arquitecto y el equipo definen en vivo un **C4 nivel contenedor** en una reunión remota.
- ✓ Queda guardado en la nube, con historial de versiones.



Lucidchart



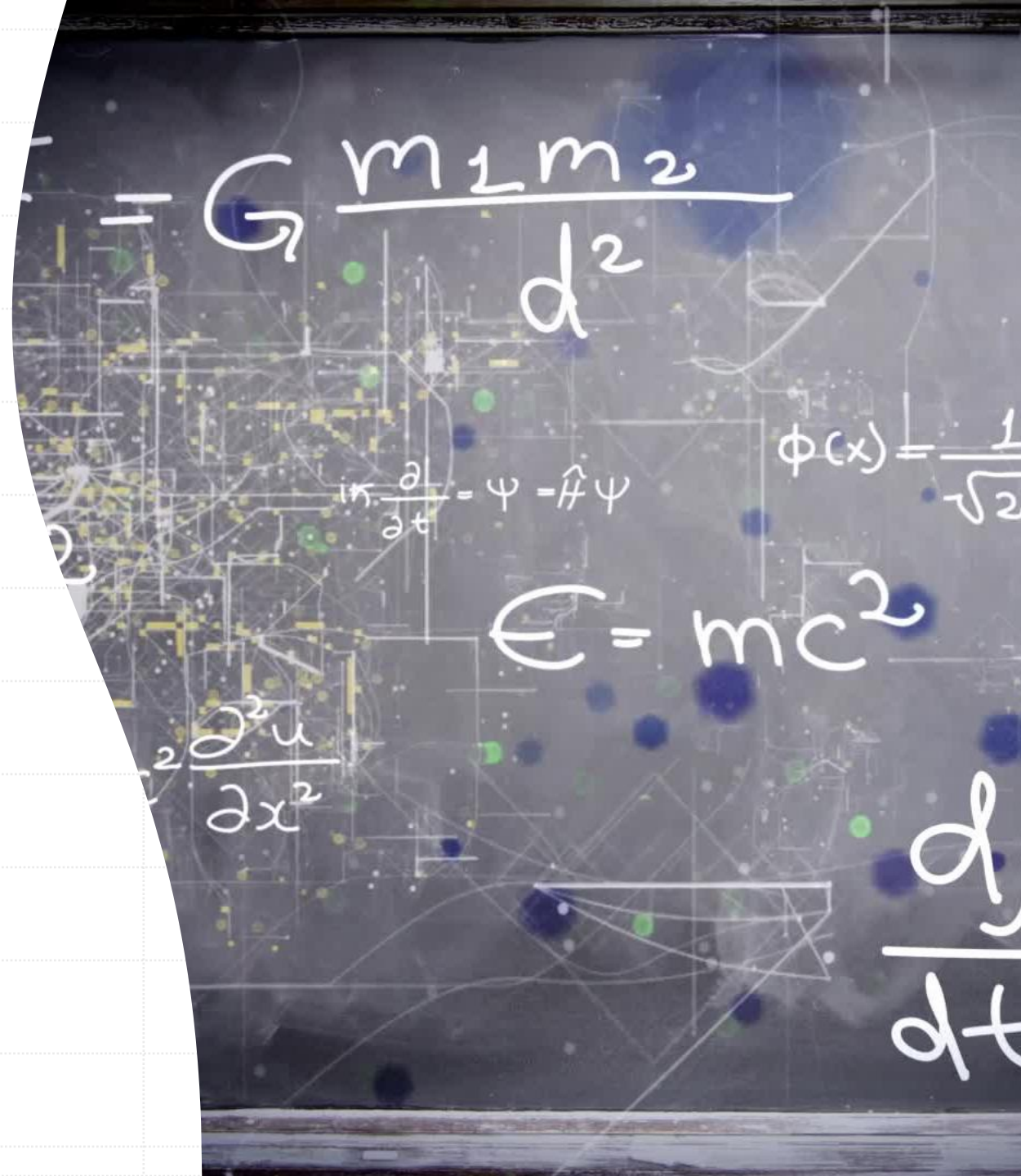
Structurizr

Qué es:

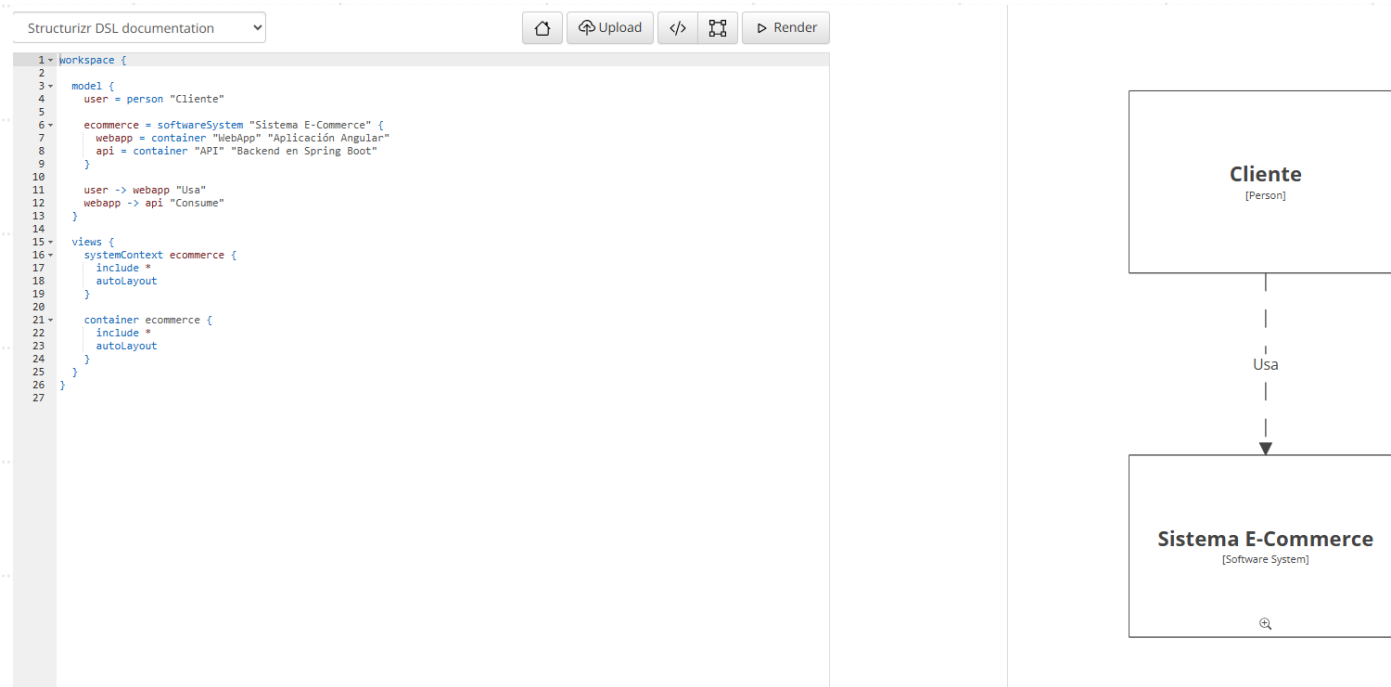
Herramienta creada por Simon Brown (autor del modelo C4).

Permite **modelar sistemas en los 4 niveles C4**:

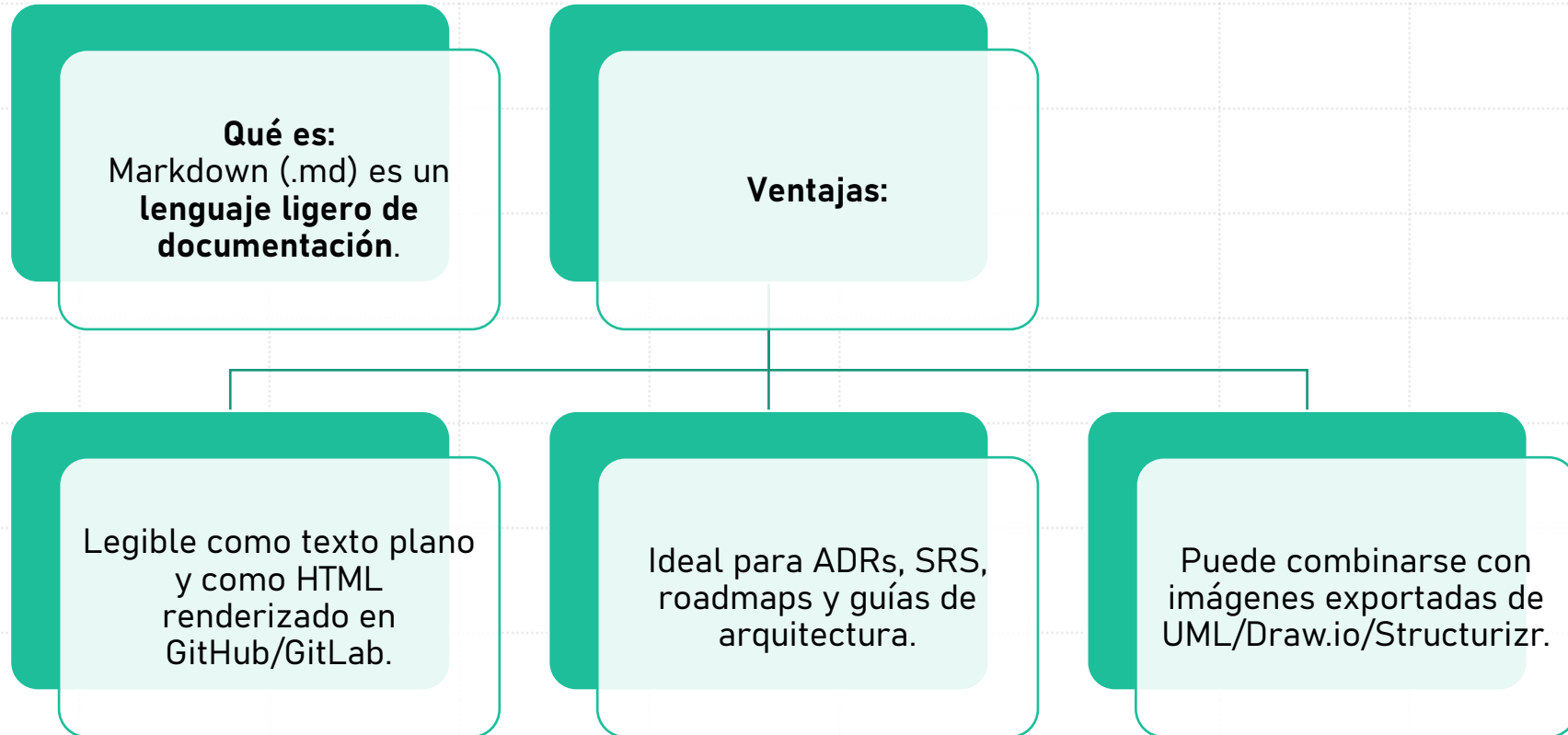
- ✓ Contexto
 - ✓ Contenedores
 - ✓ Componentes
 - ✓ Código
- **Ventajas:**
- ✓ Soporte nativo para C4 (no solo UML).
 - ✓ Diagramas siempre sincronizados con el modelo.
 - ✓ Tiene versión SaaS y también on-premise.



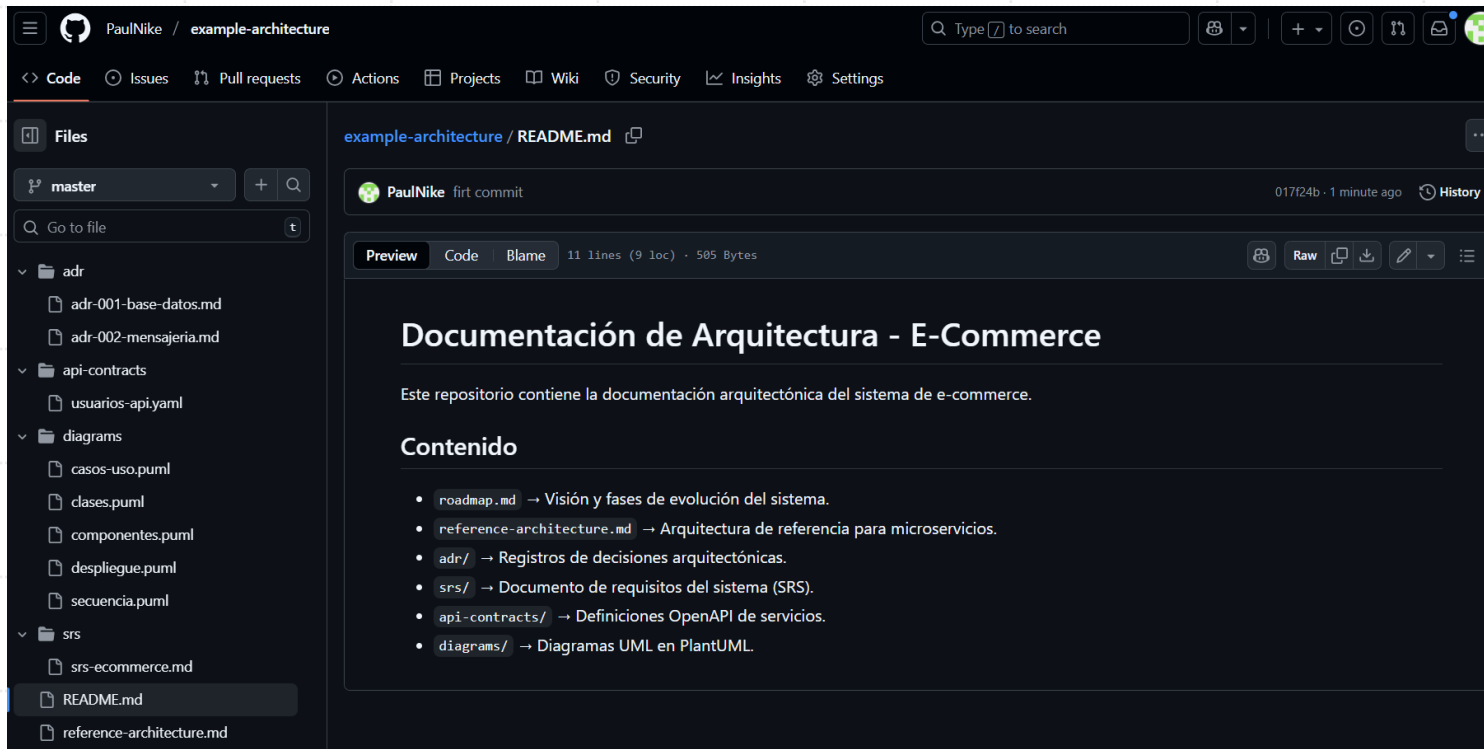
Ejemplo: Structurizr



Markdown + Git



Markdown + Git



An abstract 3D graphic featuring a grid of white rectangular blocks of varying heights, arranged in a stepped pattern. Red lines are drawn across the blocks, forming a complex, interconnected network that suggests a flow or a path. The background is a light gray grid with a subtle perspective. The text "Frameworks más usados en Arquitectura de Software / Empresarial" is overlaid on the lower part of the image.

Frameworks más usados en Arquitectura de Software /
Empresarial

TOGAF (The Open Group Architecture Framework)

- **Qué es:**

Un marco **metodológico** para diseñar, planificar, implementar y gobernar arquitecturas empresariales.

- **Dónde se usa:**

- ✓ Grandes corporaciones (banca, telecom, gobierno).
- ✓ Donde TI debe estar muy alineado al negocio.

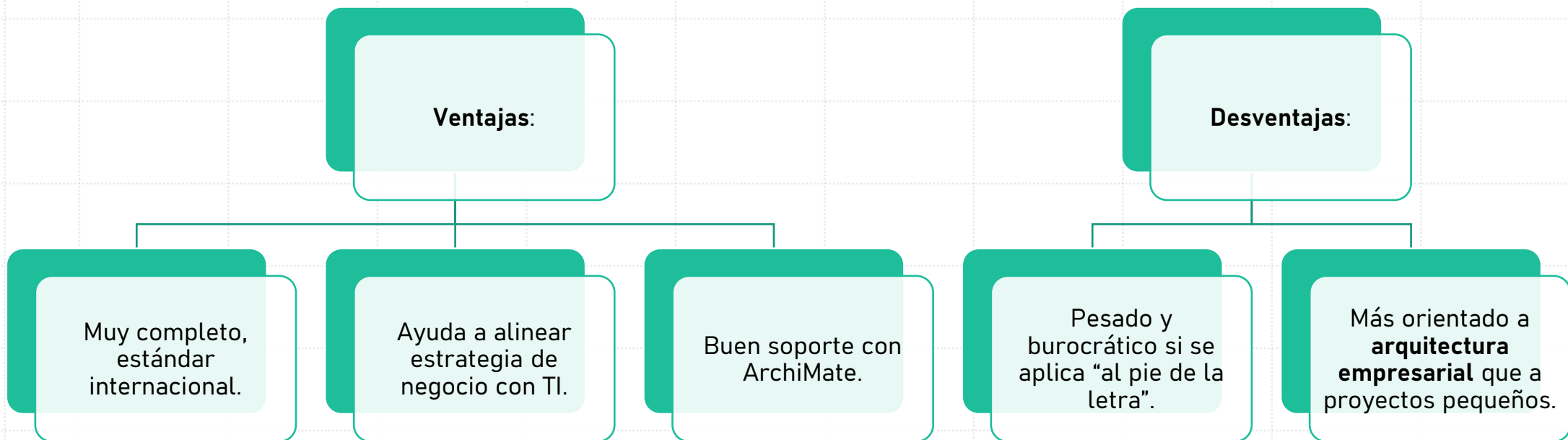




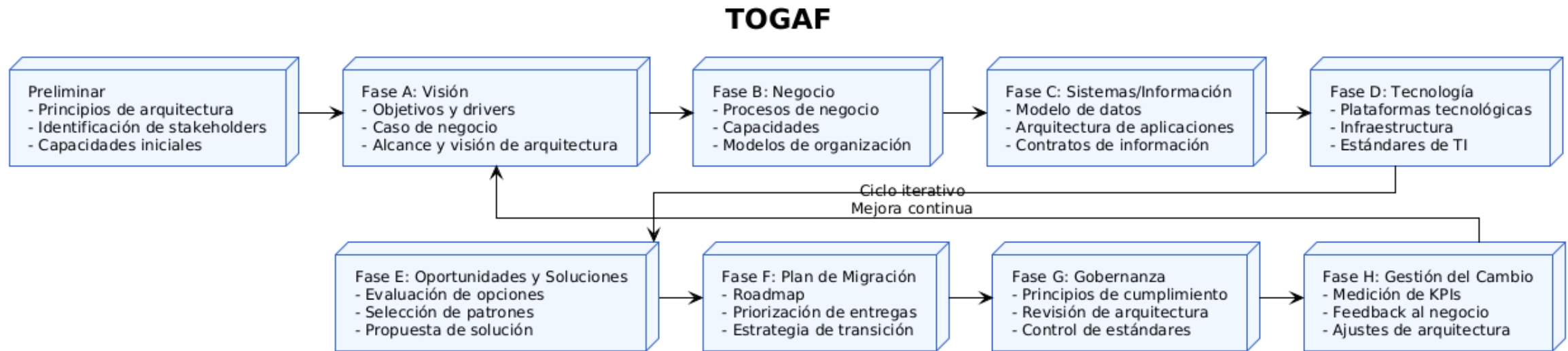
TOGAF (The Open Group Architecture Framework)

- **Componentes principales:**
 - ✓ **ADM (Architecture Development Method):** ciclo iterativo para diseñar arquitectura.
 - ✓ **Enterprise Continuum:** repositorio de artefactos, patrones y referencias.
 - ✓ **Content Framework:** define qué artefactos se producen en cada fase.

TOGAF (The Open Group Architecture Framework)



TOGAF





Zachman Framework

- ¿Qué es?: Una matriz de 6x6 que clasifica vistas de la empresa (qué, cómo, dónde, quién, cuándo, por qué).
- **Dónde se usa:** Como una matriz para clasificar y organizar artefactos (qué, cómo, dónde, quién, cuándo, por qué).

Zachman Framework

Ventajas adicionales:

Muy claro para detectar huecos en documentación.

Independiente de la tecnología → atemporal.

Útil como base teórica en universidades y formación.

Desventajas adicionales:

Poco usado en la práctica real de proyectos ágiles.

Muy conceptual, no guía implementación.

Puede generar “papeleo por papeleo” si no se adapta.

FEAF (Federal Enterprise Architecture Framework)



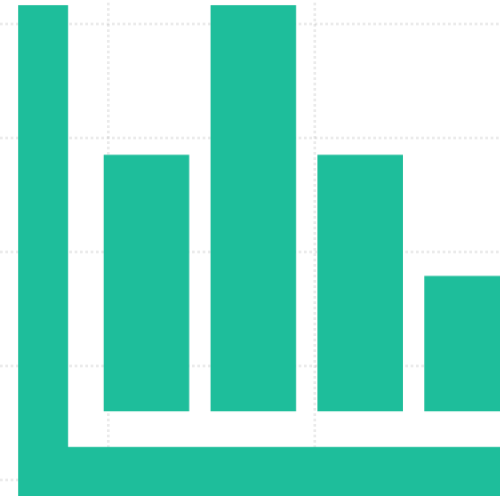
QUÉ ES: FRAMEWORK DE
ARQUITECTURA EMPRESARIAL
DEL GOBIERNO DE EE.UU.



DÓNDE SE USA: ENTIDADES
PÚBLICAS Y SECTOR
GOBIERNO.

FEAF (Federal Enterprise Architecture Framework)

- **Qué agrega:**
 - ❑ Modelos de referencia (Performance, Business, Service, Data, Technical).
 - ❑ Un **lenguaje estándar** para interoperar entre agencias.





FEAF (Federal Enterprise Architecture Framework)

Ventajas adicionales:

- Promueve interoperabilidad en organismos públicos.
- Tiene respaldo oficial, útil para auditorías y control.
- Ayuda a alinear presupuestos públicos con TI.

Desventajas adicionales:

- Poco flexible, muy orientado a sector público.
- Poco adoptado fuera de EE.UU.
- Escasa adaptación a entornos ágiles modernos.

Gartner Enterprise Architecture Framework

Qué es: Enfoque pragmático de EA de Gartner.

Qué agrega: Visión estratégica, portafolio de aplicaciones, roadmap.



Gartner Enterprise Architecture Framework

Ventajas adicionales:

- Muy orientado a **valor de negocio**.
- Lenguaje más simple que TOGAF.
- Flexible y adaptable, menos “académico”.
- Ayuda a relacionar TI con inversiones.

Desventajas adicionales:

- Menos detallado en técnicas y procesos de implementación.
- Depende mucho de la consultoría de Gartner (costosa).
- No es open source → acceso limitado a documentación.

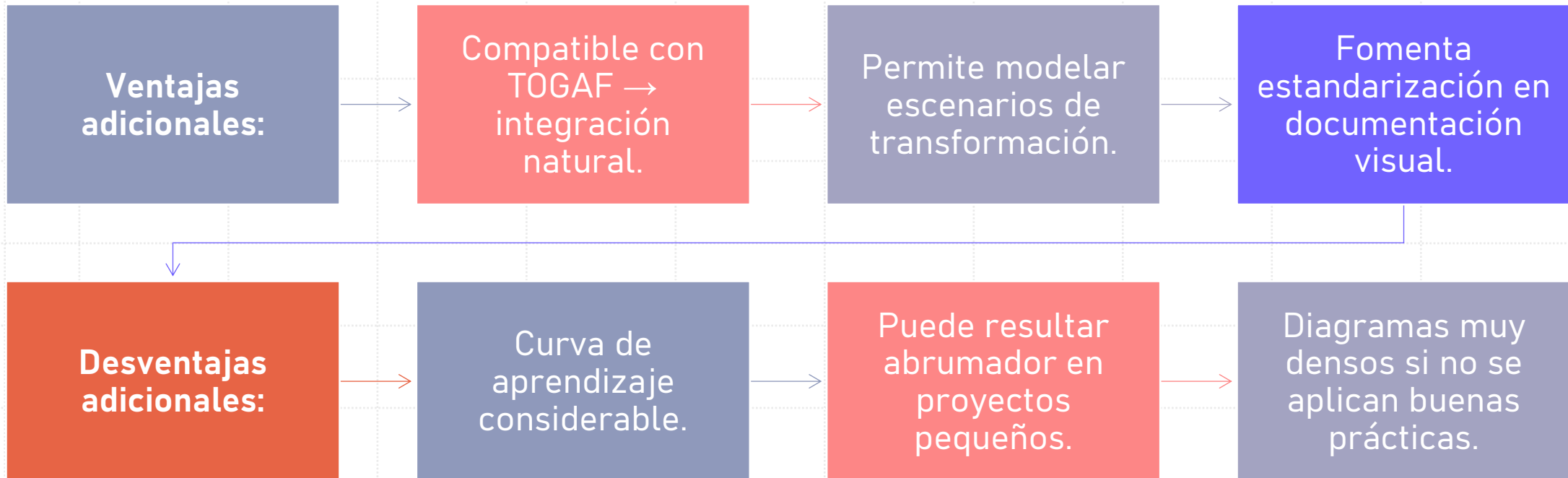


ArchiMate

Qué es: Lenguaje visual estándar para modelar arquitecturas empresariales.

Qué agrega: Diagramas claros que integran negocio, aplicación y tecnología.

ArchiMate



C4 Model



Qué es: Modelo visual jerárquico (Contexto, Contenedores, Componentes, Código).



Qué agrega: Documentación clara y simple de software.



C4 Model

Ventajas adicionales:

- Se entiende rápido, incluso por perfiles no técnicos.
- Ideal para microservicios y sistemas distribuidos.
- Puede integrarse fácilmente con Markdown, PlantUML, Structurizr.
- Compatible con “docs-as-code”.

Desventajas adicionales:

- No cubre negocio, estrategia ni gobierno.
- Puede quedarse corto en sistemas muy grandes si no se combina con otro framework.
- Requiere disciplina para mantener diagramas actualizados.

ARC42

Qué es: Plantilla para documentar arquitecturas de software (12 secciones).

Qué agrega: Estructura práctica y ligera para documentar (contexto, decisiones, calidad, riesgos, etc.).



ARC42

Ventajas adicionales:

- Muy usado en proyectos ágiles y startups.
- Compatible con C4 y docs-as-code (Markdown, AsciiDoc).
- Gratis, open source y con comunidad activa.
- Centrado en **lo que de verdad importa** en un sistema.

Desventajas adicionales:

- No cubre nivel estratégico/empresarial.
- Requiere disciplina del equipo para mantenerlo vivo.
- Menos conocido en América que en Europa.
- Puede parecer “liviano” para empresas muy reguladas.